FAR3d code

User's Guide

Main developers:

D. Spong[1] (spong@ornl.gov)

L. Garcia[2] (lgarcia@fis.uc3m.es)

J. Varela[2] (rodriguezjv@ornl.gov)

Y. Ghai[1] (ghaiy@ornl.gov)

[1] Oak Ridge National Laboratory, Oak Ridge, Tennessee 37831-8071

[2] Universidad Carlos III de Madrid, 28911 Leganes, Madrid, Spain

Terms & Conditions of Use

Far3d code is a freely distributed code under the GNU license. The developers will be grateful with the FAR3d users that include references and acknowledge the effort of setting up the code. The users are free to modify the code and to share with the Far3d community new code add-ons, that can be included in future Far3d releases.

Index

0. Quick Start

0.1 Downloading and unpacking the code

Far3d can be downloaded from https://github.com//ORNL-Fusion/FAR3d. Next, extract the files using the command:

> tar xvf FAR3d.tar

The folder FAR3d is created with the next distribution:

FAR3d /Source : code .f90 files

- → /Input_basic : default values of the input files
- → /Models : code input examples
- → /Help : help files
- → /BOOZ_XFORM : code equilibria input
- → /Addon : external accessory programs
- → User_guide.pdf : Far3d User's Guide
- → Config.sh : compiles the code and creates the user's models

0.2 Run a test case

Several test models are included with FAR3d distribution. The test cases are included in /Models folder, for example the test case "DIIID":

> cd Models/DIIID/

Inside the folder you will find the next files:

a) Input files:

Input_Model

b) Model equilibria:

Eq_DIIID_RS

c) Experimental profiles:

Data.txt

First the user should compile the code executing the bash file Config.sh (see chapter 4) and copy the executable **xfar3d** in the model folder. To run the model, execute the program:

> ./xfar3d

After the end of the run you will find in the same folder the eigenfunctions data and other output files as "farprt" file where you can find the growth rate and frequency of the instability, in this case a Toroidal Alfven Eigenmode.

0.3 Prepare your own model

You can create your own modes using the bash file "Config.sh". First, execute the bash file 'Config.sh':

> ./Config.sh

You need to introduce the name of your new model:

> Welcome to FAR3d, introduce your model name:

> 'Your model name'

> Your model was created at: cd FAR3d/Models/'Your model name'

Now, in the new folder you will find the code executable and the default input files. Modify the input file to adequate the run to your case (a description of the input files is done in the section 2). You will also need the equilibria of the model (in section 6 we explain how to create your model equilibria from a VMEC input using the program 'BOOZ_XFORM' included in Far3d distribution). In addition, you can add experimental profiles if required (see section 2 for further explanations).

1. Code introduction

1.1 FAR3d

The aim of FAR3d code is to offer a fast tool to study the linear MHD and AE stability of nuclear fusion devices. FAR3d code solves the linear reduced resistive MHD equations [1,2,3] adding the Landau damping/growth (wave-particle resonance effects) [4,5] and geodesic acoustic waves (parallel momentum response of the thermal plasma) [6]. FAR3d includes the moment equations of the energetic ion density and parallel velocity allowing treatment of linear wave-ion resonances.

The model requires Landau closure relations that can be calibrated by more complete kinetic models [6]. The simulations are based on an equilibria calculated by the VMEC code [7], but future version will include other equilibria options as SIESTA [8] or EFIT [9].

From the full MHD equations we can derive a reduced set of three equations that follow the evolution of the pressure 'p', the stream function proportional to the electrostatic potential 'Φ' and the perturbation of the poloidal flux 'ψ' retaining the toroidal angle variation 'ζ' (exact three-dimensional equilibrium [1]). The equations for the background or thermal plasma evolution are complemented by moments of the energetic ion equations to add the effect of the wave-particle interaction [10], namely the energetic particle density 'n_f' and velocity moments parallel to the magnetic field lines '$v_{||f}$'. The coefficients of the closure relation are selected to match a two-pole approximation of the plasma dispersion function.

The reduced equations are based on the following assumptions: high aspect ratio, medium thermal ß (of the order of the inverse aspect ratio ε=a/R_0, with 'a' the device minor radius and 'R_0' the major radius), small variation of the fields and small resistivity. The magnetic field and plasma velocity perturbations are defined as:

$$v = \sqrt{g} R_0 \nabla\zeta \wedge \nabla\Phi \qquad ; \qquad B = R_0 \nabla\zeta \wedge \nabla\psi$$

with $\sqrt{g}$ the Jacobian of the coordinate transformation.

1.2 Numerical model

The equations, in dimensionless form, are:

MHD equations:

[1]

$$\frac{\partial \tilde{\Psi}}{\partial t} = \sqrt{g} B \nabla_{\parallel} \Phi + \frac{\eta}{S} \tilde{J}^{\zeta} - \frac{\beta_{0e}}{2\varepsilon^2 \omega_{cy} n} \sqrt{g} B \nabla_{\parallel} p + \rho_i^2 \sqrt{\frac{\pi}{2}} \frac{v_A^2}{v_{Te}} |\nabla_{\parallel}| Q$$

[2]

$$\frac{\partial \tilde{U}}{\partial t} = -v_{eq}^{\zeta} \frac{\partial \tilde{U}}{\partial \zeta} + \sqrt{g} B \nabla_{\parallel} J^{\zeta} - \frac{\beta_0}{2\varepsilon^2} \sqrt{g} \left(\nabla \sqrt{g} \wedge \nabla \tilde{p} \right)^{\zeta} - \frac{\beta_f}{2\varepsilon^2} \sqrt{g} \left(\nabla \sqrt{g} \wedge \nabla \tilde{n}_f \right)^{\zeta} - \frac{\beta_{\alpha}}{2\varepsilon^2} \sqrt{g} \left(\nabla \sqrt{g} \wedge \nabla \tilde{n}_{\alpha} \right)^{\zeta}$$

$$- \frac{\beta_{0i}}{2\varepsilon^2 \omega_{cy}} \sqrt{g} \left[\nabla \wedge \left(\frac{\sqrt{g}}{B^2} (B \wedge \nabla p \cdot \nabla) v_{\perp} \right) \right]^{\zeta} + \omega_r \rho_i^2 \nabla_{\perp}^2 \tilde{U} - \frac{\beta_{oi}}{2\varepsilon^2 \omega_r} \tau_i p_{ieq} (S_{ei})_{imag} \Omega_d^2 (\tilde{\Phi})$$

[3]

$$\frac{\partial \tilde{p}}{\partial t} = -v_{eq}^{\zeta} \frac{\partial \tilde{p}}{\partial \zeta} + \frac{dp_{eq}}{d\rho} \frac{1}{\rho} \frac{\partial \tilde{\Phi}}{\partial \theta} - \Gamma p_{eq} \nabla \cdot v - \frac{\Gamma \beta_{0i}}{2\varepsilon^2 \omega_{cy}} \frac{p_{i,eq}}{n} \nabla \tilde{p} \cdot \nabla \wedge \frac{B}{B^2} - \frac{\Gamma p_0 B_0}{\varepsilon^2 (J + \iota I)} \left(\frac{\partial}{\partial \theta} + \frac{\partial}{\partial \zeta} \right) v_{\parallel, th}$$

[4]

$$\frac{\partial \tilde{v}_{\parallel th}}{\partial t} = -v_{eq}^{\zeta} \frac{\partial \tilde{v}_{\parallel th}}{\partial \zeta} - \frac{\beta_0}{2 n_{0,th}} \nabla_{\parallel} p$$

Energetic Particle equations:

[5]

$$\frac{\partial \tilde{n}_f}{\partial t} = -v_{eq}^{\zeta} \frac{\partial \tilde{n}_f}{\partial \zeta} - \frac{v_{th,f}^2}{\varepsilon^2 \omega_{cy}} \Omega_d (\tilde{n}_f) - n_{f0} \nabla_{\parallel} v_{\parallel f} - n_{f0} \Omega_d (\tilde{\Phi}) + n_{f0} \Omega_* (\tilde{\Phi}) + \varepsilon^2 \omega_r \omega_{cy} \frac{n_{f0}}{v_{th,f}^2} W - n_{f0} \Omega_* (W)$$

[6]

$$\frac{\partial \tilde{v}_{\parallel f}}{\partial t} = -v_{eq}^{\zeta} \frac{\partial \tilde{v}_{\parallel f}}{\partial \zeta} - \frac{v_{th,f}^2}{\varepsilon^2 \omega_{cy}} \Omega_d (\tilde{v}_{\parallel f}) - \sqrt{2} a_1 v_{th,f} |\nabla_{\parallel} \tilde{v}_{\parallel f}| - 2 a_0 \frac{v_{th,f}^2}{n_{f0}} \nabla_{\parallel} n_f + v_{th,f}^2 \Omega_* (\tilde{\Psi}) + v_{th,f}^2 \Gamma_f \Omega_* (\tilde{\Psi})$$

2nd species energetic Particle equations:

[7]

$$\frac{\partial \tilde{n}_\alpha}{\partial t} = -v_{eq}^{\zeta}\frac{\partial \tilde{n}_\alpha}{\partial \zeta} - \frac{v_{th,\alpha}^2}{\varepsilon^2 \omega_{cy,\alpha}}\Omega_d(\tilde{n}_\alpha) - n_{\alpha 0}\nabla_\parallel v_{\parallel\alpha} - n_{\alpha 0}\Omega_d(\tilde{\Phi}) + n_{\alpha 0}\Omega_*(\tilde{\Phi}) + \varepsilon^2 \omega_r \omega_{cy,\alpha}\frac{n_{\alpha 0}}{v_{th,\alpha}^2}W - n_{\alpha 0}\Omega_*(W)$$

[8]

$$\frac{\partial \tilde{v}_{\parallel\alpha}}{\partial t} = -v_{eq}^{\zeta}\frac{\partial \tilde{v}_{\parallel\alpha}}{\partial \zeta} - \frac{v_{th,\alpha}^2}{\varepsilon^2 \omega_{cy,\alpha}}\Omega_d(\tilde{v}_{\parallel\alpha}) - \sqrt{2}a_1 v_{th,\alpha}\,|\nabla_\parallel \tilde{v}_{\parallel\alpha}| - 2a_0\frac{v_{th,\alpha}^2}{n_{f0}}\nabla_\parallel n_\alpha + v_{th,\alpha}^2\Omega_*(\tilde{\Psi}) + v_{th,\alpha}^2\Gamma_\alpha\Omega_*(\tilde{\Psi})$$

Auxiliary equations:

[9] $0 = Q - \nabla_\perp^2\psi$

[10] $0 = \left(1+\rho_f^2\nabla_\perp^2\right)W_{NBI} - \rho_f^2\nabla_\perp^2\Phi$ [11] $0 = \left(1+\rho_f^2\nabla_\perp^2\right)X_{1,NBI} - \frac{\partial\psi}{\partial\zeta}$ [12] $0 = \left(1+\rho_f^2\nabla_\perp^2\right)X_{2,NBI} - \frac{\partial\psi}{\partial\theta}$

[13] $0 = \left(1+\rho_f^2\nabla_\perp^2\right)W_{\alpha} - \rho_f^2\nabla_\perp^2\Phi$ [14] $0 = \left(1+\rho_f^2\nabla_\perp^2\right)X_{1,\alpha} - \frac{\partial\psi}{\partial\zeta}$ [15] $0 = \left(1+\rho_f^2\nabla_\perp^2\right)X_{2,\alpha} - \frac{\partial\psi}{\partial\theta}$

Here, $U = \sqrt{g}\left[\nabla \wedge \left(n_0\sqrt{g}v\right)\right]^\zeta$ is the vorticity and n_0 the ion and electron mass density. The toroidal current density J^ζ is defined as:

$$J^\zeta = \frac{1}{\rho}\frac{\partial}{\partial\rho}\left(-\frac{g_{\rho\theta}}{\sqrt{g}}\frac{\partial\Psi}{\partial\theta} + \rho\frac{g_{\theta\theta}}{\sqrt{g}}\frac{\partial\Psi}{\partial\rho}\right) - \frac{1}{\rho}\frac{\partial}{\partial\theta}\left(-\frac{g_{\rho\rho}}{\sqrt{g}}\frac{\partial\Psi}{\partial\theta} + \rho\frac{g_{\rho\theta}}{\sqrt{g}}\frac{\partial\Psi}{\partial\rho}\right)$$

$v_{\zeta,eq}$ is the equilibrium toroidal rotation, $v_{\theta,eq}$ is the equilibrium poloidal rotation and $v_{||th}$ is the parallel velocity of the thermal particles. n_f and are the density of the energetic particles species normalized to the density at the n_α magnetic axis, Φ to a^2B_0/τ_{A0} and ψ to a^2B_0, with $\tau_{A0} = R_0(\mu_0 n_0)^{1/2} / B_0$ the Alfven time. All lengths are normalized to a generalized minor radius, the time to the resistive time, the magnetic field to B_0 (the averaged value at the magnetic axis) and the pressure to its equilibrium value at the magnetic axis. The Lundquist number S is the ratio of the resistive time to the Alfven time. ι is the rotational transform, $v_{th,f} = \sqrt{T_f / m_f}$ the energetic particle thermal velocity normalized to the Alfven velocity in the magnetic axis and ω_{cy} the energetic particle cyclotron frequency normalized to the Alfven time. q_f is the charge, T_f the temperature and m_f the mass of the energetic particles. I is the electric current and J the current density

The Ω_{dr} operator is constructed to model the average drift velocity of a passing particle:

$$\Omega_d = -\frac{v_{th,f}^2 v_A}{\omega_{cy} R_0}\left(\Omega_{dr}\frac{\partial}{\partial\rho} + \Omega_{d\theta}\frac{\partial}{\partial\theta} + \Omega_{d\zeta}\frac{\partial}{\partial\zeta}\right)$$

where:

$$\Omega_{dr} = \frac{\sqrt{g}}{2\rho\varepsilon^2(J+\tau I)}\left(I\frac{\partial}{\partial\zeta}\frac{1}{\sqrt{g}} - \frac{\partial}{\partial\theta}\frac{1}{\sqrt{g}}\right)$$

$$\Omega_{d\theta} = \frac{\sqrt{g}}{2\varepsilon^2(J+\tau I)^2}\left[J\frac{\partial}{\partial\rho}(J+\tau I)\frac{1}{\sqrt{g}} - \beta_*\rho(J+\tau I)\frac{\partial}{\partial\zeta}\frac{1}{\sqrt{g}}\right]$$

$$\Omega_{d\zeta} = \frac{\sqrt{g}}{2\rho\varepsilon^2(J+\tau I)^2}\left[\beta_*\rho(J+\tau I)\frac{\partial}{\partial\theta}\frac{1}{\sqrt{g}} - I\frac{\partial}{\partial\rho}(J+\tau I)\frac{1}{\sqrt{g}}\right]$$

The Ω_* operator is constructed to model the diamagnetic drift frequency:

$$\Omega_* = \frac{T_f}{q_f B_0 a^2 \rho (J+\tau I)}\frac{1}{n_{0f}}\frac{dn_{0f}}{d\rho}\left(I\frac{\partial}{\partial\zeta} - J\frac{\partial}{\partial\theta}\right)$$

Equations [3] and [4] introduce the parallel momentum response of the thermal plasma, required for coupling to the geodesic acoustic waves accounting the geodesic compressibility in the frequency range of the geodesic acoustic mode. Equations [1] and [2] include the effect of the ion finite Larmor radius effect (third and fifth terms respectively), with ρ_i the thermal ions Larmor radius and $v_{T,i}$ the ion thermal velocity. Equation [2] introduces the effect of the electron-ion Landau damping and collisions (sixth term) with ω_r the frequency of the instability. In the equations [5] and [6] the effect of the energetic particle finite Larmour radius is added by the auxiliary equations [10 – 12] (last term). Equations [1] to [3] include the two fluid terms (last terms) with Λ the electron pressure / total pressure ratio.

We use the Boozer formulation for the flux coordinates [11] where the Jacobian of the coordinate transformation is:

$$\sqrt{g} = \frac{J - \iota I}{B^2}$$

The equations are written using the equilibrium flux coordinates (ρ , θ , ζ) with ρ the generalized radial coordinate proportional to the square root of the toroidal flux function (normalized to one at the edge) and θ the poloidal angle.

The FAR3D code uses finite differences in the radial direction and Fourier expansions in the two angular variables. The numerical scheme is semi-implicit in the linear terms. The nonlinear version uses a two semi-step method to ensure Δt^2 accuracy. The functions have two components, equilibrium and perturbation, represented as $A = A_{eq} + \hat{A}$.

Present Far3d version is serial although future code version already in test phase will implement MPI and OPENMP capabilities.

For more information about the numerical model and code implementation please see the pdf files included in the folder /Help. You will find the next documents: Acoustic_modes.pdf, Boozer_coordinates.pdf, FAR3d_basic.pdf, FLR_effects.pdf, Gyrofluid_model.pdf, MHD_model_implementation.pdf, Toroidal_rotation.pdf, MHD_model.pdf, Two_fluid.pdf and Landau_elec_ion_damp.pdf.

The model was validated on previous studies, analyzing the pressure gradient driven modes, current gradient driven modes and AE in several devices [12,29], indicating a reasonable agreement with the experimental observations.

2. Input files

Far3d input file is:

Input_Model

!!!! We suggest to the users don't modify the structure of the input files to avoid the malfunction of the code. If you need to modify the input files, you should also modify the subroutine '*inputlist*'

2.1 Input file: **Input_Model**

In the first part of the input file the user can configure the simulation indicating the number of grid point, modes and number of time steps between other main parameters. In addition, you can activate the different code modules, for example the finite Larmor radius effects or introducing experimental profiles. The list of available modules and simulation options are the following:

nstres: indicates if the run is a new run if 0 new run or a continuation if 1.

numrun: (4 digit number) run number.

numruno: (4 digits number + z) name of the previous run output.

numvac: (5 digit number) index of the run number.

nonlin: linear run if 0, non linear run if 1 (no available yet).

ngeneq: indicates the equilibrium input of the model (only VMEC available now).

eq_name: equilibrium name.

maxstp: number of simulation time steps.

dt0: simulation time step.

ldim: total number of poloidal modes (equilibrium + dynamic).

leqdim: equilibrium poloidal modes.

jdim: number of radial points.

ext_prof: include experimental profiles if 1. If 0, the user must include the profiles in the file **Input_Model_Prof.**

ext_prof_name: name of the external profile file.

ext_prof_len: number of lines in the external profile.

iflr_on: index that activates the thermal ion FLR damping effects if 1.

epflr_on: index that activates the fast particle FLR damping effects if 1.

ieldamp_on: index that activates electron-ion Landau damping effect if 1 (this only is only available if **ext_prof=1**).

twofl_on: index that activates the two fluid effects if 1.

alpha_on: index that activates a 2nd fast particle species if 1.

Trapped_on: index that activates helically trapped 1st fast particle species if 1.

matrix_out: index to generate the eigensolver input files.

m0dy: index to indicate the number of equilibrium modes that evolves in time (only in nonlinear simulations, not available yet).

In the second part of the input file the user can configure the simulation parameters, introducing the list of dynamic and equilibrium modes, energetic particle ß, diffusivities, Landau closure terms between others. The list of available model parameters are the following:

mm: list of poloidal dynamic and equilibrium modes.

nn: list of toroidal dynamic and equilibrium modes.

mmeq: list of poloidal equilibrium modes.

nneq: list of toroidal equilibrium modes.

!!!!! ATTENTION: **ldim = total number of mm and leqdim = total number of mmeq !!!!!**

ipert: different options to drive a perturbation in the equilibria. There are two options:

If ipert=0 the mode 0/0 is not perturbed.

If ipert=1 the mode 0/0 is also perturbed.

widthi: size of the perturbation.

Auto_grid_on: the grid spacing is automatic selected if 1 with:

ni=(jdim/2) – 1 ; nis=(jdim/4) + 1 ; ne=jdim/4

ni: number of points interior to the island.

nis: number of points in the island.

ne: number of points exterior to the island.

!!!!! ATTENTION: **jdim = ni + nis + ne if Auto_grid_on = 0 !!!!!**

delta: normalized width of the uniform fine grid (island).

rc: center of the fine grid (island) along the normalized minor radius.

Edge_on: activates the VMEC data extrapolation.

Edge_p: grid point from where the VMEC data is extrapolated.

gamma: adiabatic index.

s: magnetic Lundquist number.

betath_factor: this factor multiplies the thermal beta to reduce or increase its value in the simulation. It should be noted that, if the factor is large, the flux surfaces calculated by VMEC are not valid and the equilibria must be recalculated.

ietaeq: index to select the resistivity profile. There are three options:

If ietaeq=1 the resisitivity is calculated using the electron temperature.

If ietaeq=2 no radial dependency. The user should include the parameter '**etascl**'.

If ietaeq=3 the resistivity is defined by the profile $\eta = eta0\left(1+\left(\frac{r}{reta}\right)^{2etalmb}\right)^{1/etalmb}$

where '**reta**' and '**etalmb**' must be introduced by the user.

bet0_f: fast particle beta.

bet0_falp: 2nd species fast particle beta.

omcy: normalized fast particle cyclotron frequency.

omcyb: normalized bounce frequency of the helically trapped fast particles.

rbound: normalized bounce distance of the helically trapped fast particle.

omcyalp: normalized 2nd species fast particle cyclotron frequency

itime: time normalization option.

dpres: electron pressure normalized to the total pressure (two fluid effects only).

stdifp: thermal pressure eq. diffusivity.

stdifu: vorticity eq. diffusivity.

stdifv: thermal plasma parallel velocity eq. diffusivity.

stdifnf: fast particle density eq. diffusivity.

stdifvf: fast particle parallel velocity eq. diffusivity.

stdifnfalp: fast particle density eq. diffusivity.

stdifvfalp: fast particle parallel velocity eq. diffusivity.

LcA0: Landau closure 1.

LcA1: Landau closure 2.

LcA2: correction to the fast particle beta.

LcA3: correction to the ratio between fast particle thermal velocity and Alfven velocity.

LcA0alp: Landau closure 1 2nd species.

LcA1alp: Landau closure 2 2nd species.

LcA2alp: correction to the 2nd species fast particle beta.

LcA3alp: correction to the ratio between fast particle thermal velocity and Alfven velocity 2nd species.

omegar: eigenmode frequency without damping effects.

iflr: thermal ions larmor radius normalized to the minor radius.

r_epflr: energetic particle larmor radius normalized to the minor radius.

r_epflralp: 2nd species energetic particle larmor radius normalized to the minor radius.

lplots: number of eigenfunction modes included in the output files.

nprint: number of step for an output in farprt file.

ndump: number of step for an output.

DIIID_u: turn on to use the same units than TRANSP output in the external profiles (so the code assumes the input is in cm, not m).

In the third part of the input file the user can introduce in the model user defined profiles, for example the energetic particle density profile, thermal electron temperature or displace the iota profile. The list of available user defined profiles are the following:

EP_dens_on: this index activates user defined fast particle density profile if 1, defined as $n_f(j) = \frac{0.5 \cdot (1 + tanh(Adens \cdot (Bdens - r(j)))) + 0.02}{0.5 \cdot (1 + tanh(Adens \cdot Bdens)) + 0.02}$.

Adens: fast particle density profile flatness.

Bdens: location of the fast particle density profile gradient.

Alpha_dens_on: this index activates user defined 2nd species fast particle density profile if 1, defined as $n_\alpha(j) = \frac{0.5 \cdot (1 + tanh(Adensalp \cdot (Bdensalp - r(j)))) + 0.02}{0.5 \cdot (1 + tanh(Adensalp \cdot Bdensalp)) + 0.02}$.

Adensalp: 2nd species fast particle density profile flatness

Bdensalp: location of the 2nd species fast particle density profile gradient

EP_vel_on: this index activates the used defined 1st species fast particle of $v_{th,f} / v_{A0}$ profile if 1.

Alpha_vel_on: this index activates the user defined 2nd species fast particle $v_{th,\alpha} / v_{A0}$ profile if 1.

q_prof_on: the safety factor profile from external profiles if 1.

Eq_vel_on: the thermal plasma eq. toroidal velocity from external profiles if 1.

Eq_velp_on: this index activates the external profile of the equilibrium poloidal rotation profile if 1.

Eq_Presseq_on: this index activates the external profile of the equilibrium thermal pressure profile if 1.

Eq_Presstot_on: this index activates the external profile of the equilibrium thermal + EP pressure profiles if 1.

deltaq: this parameter introduces a displacement in the safety factor profile (only tokamak eq.).

deltaiota: this parameter introduces a displacement in the iota profile (only stellarator eq.).

etascl: flat resistivity value.

eta0: user defined resistivity profile parameter.

reta: user defined resistivity profile parameter.

etambl: user defined resistivity profile parameter.

cnep: user defined thermal plasma density profile. 11 index should be included to define the profile $n_0(j) = cnep(0) * r^{2 \cdot cnep(i)}(j)$, with i=1,10.

ctep: user defined thermal electron plasma temperature profile. 11 index should be included to define the profile $T_e(j) = ctep(0) * r^{2 \cdot ctep(i)}(j)$, with i=1,10.

cnfp: user defined energetic particles density profile. 11 index should be included to define the profile $n_f(j) = cnfp(0) * r^{2 \cdot cnfp(i)}(j)$, with i=1,10.

cvep: user defined thermal ions parallel velocity profile (only for thermal ion FLR effects). 11 index should be included to define the profile $T_i(j) = cvep(0) * r^{2 \cdot cvep(i)}(j)$

, with i=1,10.

cvfp: user defined energetic particles parallel velocity profile. 11 index should be included to define the profile $v_{th,f} / v_{A0}(j) = cvfp(0) * r^{2 \cdot cvfp(i)}(j)$, with i=1,10.

cnfpalp: user defined 2nd species energetic particles density profile. 11 index should be included to define the profile $n_\alpha(j) = cnfpalp(0) * r^{2 \cdot cnfpalp(i)}(j)$, with i=1,10.

cvfpalp: user defined 2nd species energetic particles parallel velocity profile. 11 index should be included to define the profile $v_{th,\alpha} / v_{A0}(j) = cvfpalp(0) * r^{2 \cdot cvfpalp(i)}(j)$, with i=1,10.

eqvt: user defined thermal plasma equilibrium toroidal velocity. 11 index should be included to define the profile $v_{\zeta,eq} / v_{A0}(j) = eqvt(0) * r^{2 \cdot eqvt(i)}(j)$, with i=1,10.

eqvp: user defined thermal plasma equilibrium poloidal velocity. 11 index should be included to define the profile $v_{\zeta,eq} / v_{A0}(j) = eqvt(0) * r^{2 \cdot eqvt(i)}(j)$, with i=1,10.

2.2 Continue a simulation:

To continue a simulation you should modify three elements in the input file **Input_Model.** First **nstres = 1**, to indicate that the run is a continuation. Second you should introduce the index name (**numrun**) of the continuation run and indicate the output restart file name of the previous run (**numruno**), that has the format '*fsXXXXz*' with *XXXX* the **numrun** index of the previous run. For example, if **numrun = 0000** is the run index of the previous run, the output restart file name is **fs0000z**, so to continue this run we introduce a new run index, for example **numrun = 0001** and we the name of the output restart file name **numruno = 0000z.**

3. Experimental profiles

The input file **Input_Model** includes the option to activate experimental profiles in the simulation if **ext_prof=1.** The user must introduce the name of the file **ext_prof_name** and the length of the data rows **ext_prof_len**:

First, there is a file **header** that includes several plasma parameters:

PLASMA GEOMETRY

Vacuum Toroidal magnetic field at R=1.69550m [Tesla]

2.036657

Geometric Center Major radius [m]

1.752689

Minor radius [m]

0.602

Avg. Elongation

1.80556

Avg. Top/Bottom Triangularity

0.490581

Main Contaminant Species

12C

Main Ion Species mass/proton mass

2.0

TRYING TO GET TO BETA(0)=0.05666 , Rmax=1.752689

Next, a list of columns with the experimental data (from left to right):

Rho: the normalized square value of the toroidal flux

q: safety factor

Beam Ion Density: the density of the energetic particles in 10^{20} m^{-3} units.

Ion Density: the thermal ion density in 10^{20} m^{-3} units.

Elec Density: the thermal electron density in 10^{20} m^{-3} units.

Impurity Density: the impurity particles density in 10^{20} m^{-3} units.

Beam Ion Effective: the temperature of the energetic particles in keV units.

Ion Temp: the temperature of the thermal ions in keV units.

Electron Temp: the temperature of the thermal electrons in keV units.

Beam Pressure1: the pressure of the energetic particles in kPa units.

Thermal Pressure: the pressure of the thermal particles in kPa units.

Equil.Pressure: the equilibrium pressure in kPa units.

Tor Rot: the plasma toroidal rotation in km/s units.

Pol Rot: the plasma poloidal rotation in km/s units.

If the user activates the module of the 2^{nd} energetic particle species **alpha_on=1** with experimental profiles, the file needs to have two extra columns, one after the **Elec Density** column and another after the **Electron Temp** column defined as:

Alpha Density: the density of the 2^{nd} energetic particle species in 10^{20} m^{-3} units.

Alpha Temp: the temperature of the 2^{nd} energetic particle species in keV units.

There is an experimental profile option in the input file **Input_Model_Param** called **DIIID_u**. If **DIIID_u=1** the user can use directly the output of TRANSP code as the experimental profiles input. In this case the densities are in 10^{13} cm^{-3} units and there are 2 extra columns after the **Equil.Pressure** column:

Zeff: the atomic number of the impurities

Tor Rot: the plasma toroidal rotation frequency in kHz units.

4. Config.sh

The Config.sh bash file creates the user's model. The structure of the bash file is the following:

```
#-----------------------------------------------------------------------------------------------------

# Compilation of FAR3d executable

#-----------------------------------------------------------------------------------------------------


echo Welcome to FAR3d, introduce your model name:

read varname

cd Models

mkdir $varname

cd ../Source


#These are generic flags and commands.

Comp="gfortran"

Exc=" xfar3d"

Opt=" -O2"

Flag=" -ffree-line-length-none"


echo The FAR3d executable is been compiled, please wait...


#Main program files

OBJS=" Main.f90 Modules.f90"
```

#Setup subroutines files

OBJS2=" inputlist.f90 dfault.f90 setup.f90 setmod.f90 seteq.f90 etachi.f90 grid.f90 pert.f90 vmec.f90 ae_profiles.f90"

#Model subroutines

OBJS3=" b2lx.f90 block.f90 linstart.f90 lincheck.f90 linstep.f90"

#Operator subroutines

OBJS4=" clgam.f90 derivatives.f90 eqdy_dyeq.f90 om.f90 dlstar.f90 eigensolver_tools.f90 Laundamp_tools.f90 dlsq.f90"

#Output subroutines

OBJS5=" endrun.f90 energy.f90 output.f90 wrdump.f90 rddump.f90"

#Tool subroutines

OBJS6=" mult.f90 cnvt.f90 decbt.f90 solbt.f90 quadq.f90 functions.f90 mmlims.f90 numinc.f90 eqsplns.f90 elapsed_time.f90"

#Code compilation

$Comp -o $Exc $Opt $Flag $OBJS $OBJS2 $OBJS3 $OBJS4 $OBJS5 $OBJS6

mv xfar3d ../Models/$varname/xfar3d

cd ../Input_basic

cp * ../Models/$varname

echo Your model was created at: cd FAR3d/Models/$varname

#----------

The main elements in the Config.sh are:

varname: name of the model introduced by the use during the bash execution.

Comp: the compiler (gfortran by default).

Exc: the name of the Far3d executable (xfar3d by default).

Opt: Optimization flag (-O2 by default).

Flag: Other compiler flags (-ffree-line-length-none by default).

OBJS: List of f.90 files required to compile the executable divided in groups

If the user wants to change the compiler or flags the Config.sh must be modified accordingly.

After Config.sh bash execution a new model it will created inside the folder /Model with the name given by the user /Model/$ **varname.**

!!!! We suggest to the users don't modify the structure of the Config.sh bash, particularly the list of **OBJS,** because the consequence will be a compiler malfunction. If the users develop a new subroutine, it should be included in the list and properly referenced by the rest of code elements.

5. Code output

The Far3d outputs consist is the following files:

farprt: Main FAR3d output file.

profiles.dat: normalized profiles used in the run.

profiles_ex.dat: experimental profiles used in the run (only if **ext_prof=1**).

egn_eval_1.out: Mode matrix.

br: radial magnetic field eigenfunction.

bth: poloidal magnetic field eigenfunction.

phi: stream function proportional to the electrostatic potential eigenfunction.

pr: pressure eigenfunction.

psi: poloidal flux eigenfunction.

uzt: vorticity eigenfunction.

vr: radial velocity eigenfunction.

vth: poloidal velocity eigenfunction.

vthprlf: thermal parallel velocity eigenfunction.

nf: energetic particle density eigenfunction.

vparf: energetic particle parallel velocity eigenfunction.

nalp: 2nd species energetic particle density eigenfunction.

vparalp: 2nd species energetic particle parallel velocity eigenfunction.

spctr: modes kinetic and magnetic energy.

5.1 Output file: **farprt**

This is the main output file of FAR3d. The file structure is the following:

- Copy of the three input files.

- Copy of several parameters defined in **control**, **domain**, **equil** and **dynamo** modules.

- Mode matrix generated by the code in **setmod** subroutine

- Energy and growth rate of the modes included in the simulation each **nprint** time steps.

- Growth rate and frequency of the instability for each mode. If the code is converged, all modes must have a similar value.

5.2 Output file: **profiles.dat**

This file includes the main simulation profiles normalized to the value in the magnetic axis. The profiles included are, from left to right:

a) If **ext_prof=0**

- **r**: normalized minor radius.

- **denseq**: thermal plasma density.

- **teeq**: thermal electron temperature.

- **nfeq**: energetic particle density.

- **dnfeqdr**: radial derivative of the energetic particle density.

- **vfova**: energetic particle thermal velocity normalized to the Alfven velocity in the magnetic axis.

- **cureq**: electric current.

- **feq**: current density.

- **pressure**: thermal plasma pressure.

- **iota**: iota profile (no normalization).

- **1/D**: inverse of the metric Jacobian for n=0 (no normalization).

- **curvature**: radial derivative of the metric Jacobian forn=0 (no normalization).

- **shear**: magnetic shear (no normalization).

- **eta**: plasma resistivity.

b) If **ext_prof=1**

- **r**: normalized minor radius.

- **dnnbi**: energetic particle density.

- **dne**: thermal plasma electron density.

- **dni**: thermal ion density normalized.

- **dnalpha**: 2nd species energetic particle density (only if **alpha_on=1**)

- **tbn**: energetic particle temperature.

- **ti**: thermal ions temperature.

- **te**: thermal electrons temperature.

- **talpha**: 2nd species energetic particle temperature (only if **alpha_on=1**)

- **vfova**: energetic particle thermal velocity normalized to the Alfven velocity in the magnetic axis.

- **cureq**: electric current.

- **feq**: current density.

- **pressure**: thermal plasma pressure.

- **iota**: iota profile (no normalization).

- **1/D**: inverse of the metric Jacobian for n=0 (no normalization).

- **curvature**: radial derivative of the metric Jacobian forn=0 (no normalization).

- **shear**: magnetic shear (no normalization).

- **eta**: plasma resistivity (no normalization).

5.3 Output file: **profiles_ex.dat**

This file includes the profiles used in the simulation. The profiles included are, from left to right:

- **r**: normalized minor radius.

- **dnnbi**: energetic particle density (m^{-3}).

- **dne**: thermal plasma electron density (m^{-3}).

- **dni**: thermal ion density normalized (m^{-3}).

- **dnalpha**: 2nd species energetic particle density (m^{-3}). Only if **alpha_on=1**.

- **tbn**: energetic particle temperature (keV).

- **ti**: thermal ions temperature (keV).

- **te**: thermal electrons temperature (keV).

- **talpha**: 2nd species energetic particle temperature (keV). Only if **alpha_on=1**.

- **vzt_eq**: equilibrium thermal plasma toroidal velocity (m/s).

- **vth_eq**: equilibrium thermal plasma toroidal velocity (m/s).

- **vthermaleq**: thermal ions = velocity (m/s).

- **eta**: plasma resistivity (m^3 kg / sC^2).

6. Create your model equilibria

FAR3d requires as input a **VMEC** equilibria transformed to Boozer coordinates. The coordinate system transformation is done by the code '**BOOZ_XFORM**', also included in the FAR3d distribution, at the folder with the same name. The structure of /BOOZ_XFORM folder is the following:

/BOOZ_XFORM → /Source : code .f90 files

→ /Equilbria : collection of equilibria input

VMEC code can be obtained from the distribution **STELLOPT**. Once installed **VMEC**, you need the input file **input.(case name)** to create the equilibria, executing the program:

> xvmec2000 input.(case name)

VMEC generates the next output files:

jxbout.(case name).txt

mercier.(case name)

threeed1.(case name)

wout_(case name).nc

The FAR3d input is generated executing **BOOZ_XFORM** using the input file **in_booz.(case_name)**:

> xbooz_xform in_booz.(case_name) far

The program **BOOZ_XFORM** creates the next files:

bmnc_booz.txt, gimnc_booz.txt, gmnc_booz.txt, grrmnc_booz.txt, grrojmnc_booz.txt, grtmnc_booz.txt, grtojmnc_booz.txt, gttmnc_booz.txt, gttojmnc_booz.txt, jbgrrmnc_booz.txt, jbgrtmnc_booz.txt, jbgttmnc_booz.txt, phi_1b.txt, phi_2b.txt, phi_3b.txt, pmns_booz.txt, prfeq_vmec.txt, R_Z_1.txt,

R_Z_1b.txt, R_Z_2.txt, R_Z_2b.txt, R_Z_3.txt, R_Z_3b.txt, rmnc_booz.txt, sum_gmncb.txt, woutb and zmns_booz.txt

The input file of FAR3d is **woutb** file. The rest of outputs are text file with info of the transformation.

The **BOOZ_XFORM** source included in the FAR3d distribution was modified respect to the Stellopt distribution. There are 5 new subroutines: boozer_metric.f, write_polcut.f, root.f, vcoords_gijb.f and boozer_gij.f. Also, the next subroutines are modified: boozer_xform.f, booz_params.f, boozer.f and allocate_boozer.

7. Code Add-ons

FAR3d add-ons are external programs of the main distribution that must be compiled and executed independently. These programs include new analysis tools using FAR3d output.

7.1 Eigensolver.

The eigensolver program Xjdqz is compiled using the script "Eigensover.sh" located at /FAR3d/Addon and requires the library libjdqz.a. The source to build this library can be obtained from:

http://www.staff.science.uu.nl/~sleij101/JD_software/jd.html.

In addition, the packages LAPACK and LBLAS should be installed in the computer.

After running the shell, the executable "xEigen" is created at /Addon/Eigensolver.

Another option is to use the two matrix files with Jacobi-Davidson solvers in the SLEPC library (http://slepc.upv.es). Some limited testing has been done with the Slepc solvers, indicating agreement with results from xjdqz and xae3d solvers. SLEPC requires the installation of PETSC (must be the same versions), and has the advantage over xjdqz that it is parallelized and can be used for very large eigenvalue problems.

To use the eigensolver follow the next steps:

a) Activate the generation of the eigensolver input: **matrix_out** parameter in **Input_Model** (.true.).

b) Execute FAR3d.

c) The eigensolver input is created in the model folder: a_matrix.dat, b_matrix.dat and jdqz.dat.

d) The next message will appear after the FAR3d execution:

```
================================================
=======FAR3d Eigensolver module activated========
================================================
Execute xEigen located at /FAR3d/Addon/Eigensolver
================================================
==./../../xEigen frq grwth=======================
==frq : reference normalized mode frequency======
==grwth : reference normalized growth rate=======
================================================
```

This message indicates that the eigensolver input is created.

e) Execute the eigensolver program from the model folder:

./../../../Addon/Eigensolver/xEigen (normalized frequency) (normalized growth rate).

The eigensolver output are two files:

egn_values.dat: file with two columns, first the normalized mode growth rate and second the normalized mode frequency

egn_mode_asci.dat: file with one column, number of modes calculated, number of poloidal modes /2, number of grid points, list of poloidal number and toroidal numbers, radial grid points, list of Φ eigen-functions.

To create a set of files where the data of the simulation is divided in different mode target where the poloidal modes are ordered in different columns, the program columns.f90 is included in the eigensolver folder. It requires the file **egn_mode_asci.dat**

a) Compile the program columns.f90 and create the executable c_AE.

b) Run c_AE.

7.2 Graphical module.

The graphical module transforms FAR3d output to the vtk format. The vtk format is a standard used by programs as VISIT and Paraview.

To use the graphical module follow the next steps:

Copy the program vtk_format.f90 and the input file Input_GM in your folder of your model. You can find vtk_format.f90 and Input_GM at:

/FAR3d/Addon/Graphics_module/

Compile vtk_format.f90 and execute. vtk_format.f90 execution requires the equilibrium file, dump file and input file of the model (Input_model) as well as the input file Input_GM.

These are the variables of vtk_format.f90 included in the input list Input_GM:

name_vtk: file name

nx_vtk: number of point in X

ny_vtk: number of point in Y

nz_vtk: number of point in Z

pr_vtk: if 1, pr vtk output is activated

psi_vtk: if 1, psi vtk output is activated

phi_vtk: if 1, phi vtk output is activated

uzt_vtk: if 1, uzt vtk output is activated

nf_vtk: if 1, nf vtk output is activated

vprlf_vtk: if 1, vprlf vtk output is activated

vthprlf_vtk: if 1, vthprlf vtk output is activated

vr_vtk: if 1, vr vtk output is activated

vth_vtk: if 1, vth vtk output is activated

Br_vtk: if 1, Br vtk output is activated

Bth_vtk: if 1, Bth vtk output is activated

nalp_vtk: if 1, nalp vtk output is activated

vprlalp_vtk: if 1, vprlalp vtk output is activated

preq_vtk: if 1, pr vtk output is activated (only non linear)

nfeq_vtk: if 1, nf vtk output is activated (only non linear)

Vvec_vtk: if 1, velocity vector vtk output is activated (only non linear)

Bvec_vtk: if 1, magnetic field vector vtk output is activated (only non linear)

8. Code variable Index

MODULE VARIABLES:

cotrol

numhist is the simulation name you want to continue

numrun and numruns are the run number

numruno is the previous run number

numvac is the equilibrium number

stdifp is the pressure diffusivity

stdifu is the vorticity diffusivity

stdifv is the parallel velocity diffusivity

stdifnf is the fast particle density diffusivity

stdifvf is the fast particle parallel velocity diffusivity

stdifnfalp is the fast particle second species density diffusivity

stdifvfalp is the fast particle second apscies parallel velocity diffusivity

dt0 is the time step

Adens is the fast particle density flatness (if no external profiles)

Bdens is the location of the fast particle density gradient (if no external profiles)

Adensalp is the fast particle 2nd species density flatness (if no external profiles)

Bdensalp is the location of the fast particle 2nd species density gradient (if no external profiles)

LcA0 Landau closure 1

LcA1 Landau closure 2

LcA2 correction to the fast particle beta

LcA3 correction to the ratio between fast particle thermal velocity and Alfven velocity

LcA0alp Landau closure 1 2nd fast particle species

LcA1alp Landau closure 2 2nd fast particle species

LcA2alp correction to the fast particle beta 2nd fast particle species

LcA3alp correction to the ratio between fast particle thermal velocity and Alfven velocity 2nd fast particle species

ext_prof introduce the experimental profiles using an external text file

omegar eigenmode frequency (required to introduce the damping effects)

iflr thermal ions Larmour radius normalized to the device minor radius

r_epflr fast particle Larmour radius normalized to the device minor radius

r_epflralp 2nd species fast particle Larmour radius normalized to the device minor radius

dpres electron pressure normalized to the total pressure (required in the two fluid approximation)

DIIID_u activates external profiles using TRANSP code units (cm)

betath_factor thermal beta factor (if 1 the same value than the equilibria)

etascl user defined constant value for the resistivity (if ietaeq=2)

deltaq user defined safety factor profile displacement

deltaiota user defined iota profile displacement

r**eta,eta0,etalmb** user defined resistivity profile (if ietaeq=3)

ihist previous run number

nocpl if zero the run doesn't include couplings

maxstp number of time steps in the run

nstep and **nstep1** internal loop time step number

ndump indicates the number of step per eigenfunctions output

nprint indicates the number of step per output in farprt output

lplots number of modes in the eigenfunctions output

itime time step normalization option

noeqn number of equations of the model

edge_p grid point from where the VMEC data is extrapolated

inalp,ivalp,iq,iw,ix1,ix2,iwa,ix1a,ix2a internal variables

iflr_on activates thermal ion FLR effects if 1

epflr_on activates energetic particle FLR effects if 1

ieldamp_on activates electron-ion Landau damping if 1

twofl_on activates two fluids effects if 1

alpha_on activates a 2nd energetic particle species if 1

EP_dens_on activates user defined density profiles of energetic particle species if 1

Alpha_dens_on activates user defined density profiles of 2nd species energetic particle species if 1

EP_vel_on user defined fast particle vth/vA0 profile

Alpha_vel_on user defined 2nd species fast particle vth/vA0 profile

eq_name name of the equilibria

ext_prof_name external profile file name

ext_prof_len number of lines in the external profile file

deltaq safety factor displace

deltaiota iota displace

matrix_out create the input of the eigensolver (not available yet)

leqdim is the number of equilibrium modes

ldim is the total number of modes (equilibrium + dynamic)

jdim is the number of radial point

nstres indicates if the run is new (if 0) or a continuation (if 1)

Eq_vel_on activates external profiles for the equilibrium toroidal velocity if 1

Eq_velp_on activates external profiles for the equilibrium poloidal velocity if 1

Eq_Presseq_on activates external profiles for the thermal pressure if 1

Eq_Presstot_on activates external profiles for the thermal and EP pressures if 1

Auto_grid_on activates a default grid spacing

Edge_on activates the VMEC data extrapolation

domain

mjeq number of radial grids in the equilibrium file

nfp indicates the number of field periods of the equilibrium

rfar normalized radius in the equilibrium file

qfar safety factor in the equilibrium file

mj last point of the radial grid

lmax total number of modes (input)

leqmax total number of modes (input)

lbmax total number of modes in the equilibrium file

mmin smallest poloidal mode for each toroidal mode family

mmax largest poloidal mode for each toroidal mode family

nmin smallest toroidal mode

nmax largest toroidal mode

mmineq smallest equilibrium poloidal mode for each equilibrium toroidal mode family

mmaxeq largest equilibrium poloidal mode for each equilibrium toroidal mode family

nmineq smallest equilibrium toroidal mode

nmaxeq largest equilibrium toroidal mode

mjm1 is mj-1

mjm2 is mj-2

lmax0 total number of equilibrium poloidal modes

lmaxn total number of dynamic poloidal modes

l0 indicates the (0,0) mode

leq0 indicates the (0,0) equilibrium mode

lhmax indicate the helicity number of dynamic modes

lheqmx indicate the helicity number of equilibrium modes

nnum indicates the toroidal mode order number

m0dy indicates the number of evolving equilibrium modes (not available, only non linear)

mxmband number of poloidal modes for each toroidal family including both parities

nnd number of n-values with one parity

nst number of n-values with both parity

mminb smallest poloidal mode in the equilibrium file

mmaxb largest poloidal mode in the equilibrium file

nminb smallest toroidal mode in the equilibrium file

nmaxb smallest poloidal mode in the equilibrium file

mxmbandb number of poloidal modes for each toroidal family including both parities in the equilibrium file

mmaxx absolute value of the largest poloidal mode

mmaxxb absolute value of the largest poloidal mode in the equilibrium file

mbandeq number of equilibrium poloidal modes for each toroidal family including both parities

mm Vector with the dynamic polidal mode numbers

nn Vector with the dynamic toroidal mode numbers

mh Vector with the helicities dynamic polidal mode numbers

nh Vector with the helicities dynamic toroidal mode numbers

mmeq Vector with the equilibrium polidal mode numbers

nneq Vector with the equilibrium toroidal mode numbers

mheq Vector with the helicities equilibrium polidal mode numbers

nheq Vector with the helicities equilibrium toroidal mode numbers

mmb Vector with the dynamic polidal mode numbers in the equilibrium file

nnb Vector with the dynamic toroidal mode numbers in the equilibrium file

mmstart lower bound of m values

mmend upper bounds of m values

mmstartb lower bound of m values in the equilibrium file

mmendb upper bounds of m values in the equilibrium file

r is the normalized minor radius

rinv is the inverse of the normalized minor radius

dc1m,dc1p,dc2m,dc2p,del2cm,del2cp are rerivate weigths to be used in grid subroutine

wt1m,wt10,wt1p,wt2m,wt20,wt2p are derivative weights to be used in blockj, block0, b2lx and b2lx0 subroutines

ll matrix with the dynamic totoidal and poloidal mode index

llb matrix with the dynamic totoidal and poloidal mode index in the equilibrium file

lleq matrix with the equilibrium totoidal and poloidal mode index

m1n lower poloidal mode in the mode vector

mrang rank of the mode vector

ll0 index of the equilibrium poloidal mode

lln index of the equilibrium and dynamic poloidal mode

lo index of the equilibrium poloidal mode if no couplings

llno index of the dynamic poloidal mode in linstart subroutine

signl vector with the sign of the dynamic modes

signleq vector with the sign of the equilibrium modes

lnumn indicates the total number of equilibrium and dynamic poloidal modes mnumn indicates the total number of equilibrium and dynamic poloidal modes

ni number of points interior to the island

nis number of points in the island

ne number of points exterior to the island

delta width of the uniform fine grid (island)

rc center of the fine grid (island)

fti fraction of interior point in transition region

fte fraction of exterior points in transition region

bmodn normalized module of the magnetic field

mu0 vacuum magnetic permeability (MKS)

uion fast particle Z number

vthi normalized ion thermal velocity at the magnetic axis

vthe normalized electron thermal velocity at the magnetic axis

xnuelc0 electron-ion collision FR axis (MKS)

coul_log Coulomb logarithm (Te > 10 eV)

dnnbi normalized fast particle density

dne normalized thermal electron density

dni normalized thermal ion density

pthermal normalized fast particle temperature

ti normalized thermal ion temperature

te normalized thermal electron temperature

vzt_eqp normalized equilibrium toroidal rotation (to the Alfven velocity)

vth_eqp normalized equilibrium poloidal rotation (to the Alfven velocity)

qprofile safety factor

dnnbinn fast particle density (MKS)

dnenn thermal electron density (MKS)

dninn thermal ion density (MKS)

tinn thermal ion temperature (eV)

tenn thermal electron temperature (eV)

tbn normalized fast particle temperature

tbnnn fast particle temperature (eV)

pthermalnn normalized thermal pressure

vthermalep thermal velocity of the fast particles (MKS)

vAlfven normalized Alfven velocity

vtherm_ionP normalized thermal ions velocity

dnalpha normalized density of the 2nd fast particle species (MKS)

dnalphann density of the 2nd fast particle species (MKS)

talpha normalized temperature of the 2nd fast particle species

talphann temperature of the 2nd fast particle species (eV)

pep fast particle pressure

pepnn normalized fast particle pressure

ptot equilibrium + fast particle pressure

ptot normalized equilibrium + fast particle pressure

equil

qq safety factor

qqinv inve3rse of the safety factor (iota)

qqinvp radial derivate of the safety factor inverse

denseq normalized thermal plasma density

denseqr radial derivate of the normalized thermal plasma density

psieq normalized equilibrium poloidal flux

chieq normalized equilibrium toroidal flux

preq normalized equilibrium thermal plasma pressure

feq normalized equilibrium current density

cureq normalized equilibrium electric currect

teeq normalized equilibrium plasma temperature

nfeq normalized equilibrium fast particle density

dnfeqdr radial derivate of the normalized fast particle density

vfova fast particle thermal velocity normalized to the Alfven velocity

vfova2 is vfova^2

vtherm_ion user defined normalized thermal ion velocity

vzt_eq user defined normalized equilibrium toroidal rotation (to the Alfven velocity)

vth_eq user defined normalized equilibrium poloidal rotation (to the Alfven velocity)

norm_eildump factor in the terms that include the electron-ion Landau damping effects

nalpeq normalized equilibrium density of the second fast particle species

valphaova Fast particle thermal velocity normalized to the Alfven velocity of the second fast particle species

valphaova2 is vfalphaova^2

dnalpeqdr radial derivate of the normalized density of the second fast particle species

ndevice type of device

ngeneq type of equilibria input (if 1 VMEC)

leq dummy variable

ndat indicates the number of radial points in several subroutines

eps ratio of the minor and major radius

bet0 thermal beta

omcy cyclotron frequency of the fast particles (normalized to the Alfven time)

omcyalp cyclotron frequency of the 2nd species of fast particles (normalized to the Alfven time)

omcyb bounce frequency of the helically trapped energetic particles (normalized Alfven time)

rbound bounce distance of the helically trapped energetic particles (normalized to the major radius)

bet0_f fast particle beta

bet0_alp 2nd species fast particle beta

rs eigenfunction radial width

xr dummy value for the radial point in several subroutines

xdat dummy value

nstep_count time step screen out

cnep thermal plasma density

ctep thermal plasma temperature

cnfp fast particle density

cvfp fast particle parallel velocity

cvep thermal ions parallel velocity

cnfpalp 2nd species fast particle density

cvfpalp 2nd species fast particle parallel velocity

eqvt normalized equilibrium toroidal rotation (to the Alfven velocity)

eqvp normalized equilibrium poloidal rotation (to the Alfven velocity)

grr,grt,gtt,grz,gtz,gzz covariant metric elements with r=radial, t=poloidal, z=toroidal

sqgi inverse of the metric Jacobian

sqg metric Jacobian

grroj,grtoj,gttoj,grzoj,gtzoj,gzzoj covariant metric elements divided by the Jacobian

grrup,grtup,grzup,gttup,gtzup,gzzup contravariant metric elements

bmod normalized magnetic field module (to the value in the mag. axis)

bst = -rinv*(d(preq)/dr)*bet0*sd2*sqg(:,l)/(2*(mmb(l)*qqinv-nnb(l)))

jbgrr = sqg*grr

jbgrt = sqg*grt

jbgtt = sqg*gtt

jbgrr = sqg*grr

jbgrz = sqg*grz

jbgtz = sqg*gtz

omdr,omdt,omdz Omega d operator (r, th and zt components)

omdrprp,omdt[prp,omdzprp Omega d operator helically trapped particles (r, th and zt components)

djroj = sqgi*[d/dr (sqg)]

djtoj= (sqgi/rho)*[d/dth (sqg)]

djzoj= sqgi*[d/dzt (sqg)]

dbsjtoj= sqgi*[d/dth (sqg*bst)]

dbsjzoj= sqgi*[d/dzt (sqg*bst)]

dbsjtbj= sqg*[d/dzt (sqg*bst)]

dgttr= sqg*[d/dr (gtt)]

dgrrt= (sqg/rho)*[d/dth (grr)]

dgrtt= (sqg/rho)*[d/dth (grt)]

dgttt= (sqg/rho)*[d/dth (gtt)]

dgrrz= sqg*[d/dzt (grr)]

dgrtz= sqg*[d/dzt (grt)]

dgttz= sqg*[d/dzt (gtt)]

dgrtp= sqgi*[(d/dzt - qqinv*d/dth)(grt)]

dgttp= sqgi*[(d/dzt - qqinv*d/dth)(gtt)]

jsq= sqg*sqg

bsgrt= bst*grt*sqgi

bsgtt= bst*gtt*sqgi

bsq= bst*bst

bsqgtt= bst* bst*gtt*sqgi

sqgdroj = sqgi*d(sqg)/dr

sqgdthoj = (sqgi/rho)*d(sqg)/dth

sqgdztoj = sqgi*d(sqg)/dzt

sqgdztojbst = bst*sqgdztoj

sqgdthojbst = bst*sqgdthoj

sqgibmodith = bmod*sqg*[d/dth (sqgi)] / rho

sqgibmodizt = bmod*sqg*[d/dtz (sqgi)]

eildrr,eildrt,eildrz,eildtt,eildtz,eildzz,eildr,eildt,eildz electron-ion Landau damping terms

lplrr,lplrt,lplrz,lplr,lpltt,lpltz,lplt,lplz,lplzz perpendicular gradient operator terms

dynamo

ietaeq index to select the type of eta (if 1 the electron temperature is used)

ipert index of the perturbation type

dt time step

dtd2 = dt/2

time indicates the simulation time

s magnetic Lundquist number

gamma polytropic index

pertscl perturbation scale factor

eta magnetic diffusivity

widthi size of the perturbation

gammai perturbation

cmamm,cmamp,cmapm,cmapp sign matrix

amat,bmat,cmat tridiagonal matrix where the model terms are stored using block subroutine

amatw,bmatw,cmatw,amatwalp,bmatwalp,cmatwalp tridiagonal matrix where the model terms are stored using block subroutine

xt,yt,xw matrix where the model terms are stored using b2lx subroutine

ipc,ipcw,ipcwalp dummy matrix used in solbt subroutine

psi poloidal flux (norm minor radius^2 * mag. axis magnetic field)

phi electrostatic potential (norm minor radius^2 * mag. axis magnetic field / resistive time)

pr pressure (norm pressure in the mag. axis)

uzt vorticity (toroidal component, norm Alfven time)

nf fast particle density (norm fast particle density in the mag. axis)

vprlf fast particle parallel velocity (norm to the Alfven time / major radius)

vthprlf thermal parallel velocity (norm to the Alfven time / minor radius)

nfalp density of the second fast particle species (norm 2ndfast particle density in the mag. axis)

vprlfalp parallel velocity of the second fast particle species (norm to the Alfven time / major radius)

scratch

sc1,sc2,sc3,sc4,sc5,sc6,sc7,sc8 dummy dynamic variables with radial and angular dependency

sceq1,sceq2,sceq3,sceq4,sceq5,sceq6,sceq7 dummy equilibrium variables with radial and angular dependency

sd1,sd2,sd3,sd4,sd5,sd6 dummy variables with only radial dependency

SUBROUTINE VARIABLES:

ae_profiles

j,i,l,je dummy index

xx,bspl,cspl,dspl dummy vector

xpi 4.*atan(1.)

B0_e input, Vacuum Toroidal magnetic field [T]

R0_e input, Geometric Center Major radius [m]

a_e input Minor radius [m]

uion_e input**,** Main Ion Species mass/proton mass

kappa_e input**,** Avg. Elongation

delt_e input**,** Avg. Top/Bottom Triangularity

beta0_e input, total beta

rmax_e input, major radius

etactte ctte resistivity

qe electron charge (C)

me electron mass (Kg)

epsil vacuum permittivity (F/m)

kblotz Boltzmann ctte (Conversion factor eV to J)

ns0 number of rows input file

nunit unit number of the input file

rho_e input, normalized minor radius

qprof input, safety factor

den_beam_e input, EP density radial profile [10^{20} m^{-3}]

den_ion_e input, thermal ion radial density profile [10^{20} m^{-3}]

den_elec_e input, thermal electron radial density profile [10^{20} m^{-3}]

den_imp_e input, impurity radial density profile [10^{20} m^{-3}]

temp_beam_e input, EP radial energy profile [keV]

temp_ion_e input, thermal ion radial energy profile [keV]

temp_elec_e input, thermal electron radial energy profile [keV]

pres_beam_e input, beam radial pressure profile [kPa]

pres_thermal_e input, thermal plasma radial pressure profile [kPa]

pres_equil_e input, total radial pressure profile [kPa]

zeff_e input, Ion Species mass/proton mass profile

tor_rot_freq_e input, equilibrium radial toroidal rotation frequency profile [1/s]

tor_rot_vel_e input, equilibrium radial toroidal rotation profile [km/s]

pol_rot_vel_e input, equilibrium radial poloidal rotation profile [km/s]

den_alpha_e input, second EP population density radial profile [10^{20} m^{-3}]

temp_alpha_e input, second EP population energy radial profile [keV]

bt0 variable code, magnetic field axis [T]

rmajr variable code, major radius [m]

rminr variable code, minor radius [m]

mu0prof variable code, magnetic permeability vacuum [MKS]

va0 variable code, Alfven velocity magnetic axis [km/s]

omgcya variable code, cyclotron frequency [1/s]

betalf variable code, EP beta magnetic axis

bet00 variable code, thermal beta magnetic axis

vf0 variable code, EP beta

vtor0 variable code, normalized EP thermal velocity magnetic axis

dnnbinn_max maxima of the EP radial density profile

dnenn_max maxima of the thermal electron radial density profile

dninn_max maxima of the thermal ion radial density profile

tinn_max maxima of the thermal ion radial temperature profile

tenn_max maxima of the thermal electron radial temperature profile

tbnnn_max maxima of the EP radial temperature profile

pthermalnn_max maxima of the thermal pressure radial profile

ptotnn_max maxima of the total pressure radial profile

dnalphann_max maxima of the second EP population radial density profile

talphann_max maxima of the second EP population radial temperature profile

b2lx(tx,itx,ieqn,ivar,ith,ir,izt,coef)

blockj(tx,itx,ieqn,ivar,ith,ir,izt,coef)

tx multiplicative factor (with radial and angular dependencies)

itz multiplicative factor sign

ieqn equation index where the term is included

ivar perturbation variable index

ith number of derivation in theta

ir number of devivation in rho

izt number of derivations in zeta

coef coefficient that multiplies tx

ity variable sign (sin or cos decomposition)

iph = ity**(ith+izt)*(-1)**((ith+izt+1)/2)

ich = itx*ity

l,ln,m,n,l1,l2,l3,l3n,lp,lpn,lpp,ibnd,j,il dummy integer

energy(i)

epsi = $(\Delta r/2)(\Delta\Psi/\Delta r)^2 + (m/\Delta r)\Delta\Psi_+^2$

ephi = $(\Delta r/2)(\Delta\Phi/\Delta r)^2 + (m/\Delta r)\Delta\Phi_+^2$

epr = r*sqg*pr^2

eprnc = r*sqg*pr^2

ekenc = r*denseq*[sqg*grr*$(v^\rho)^2$ + sqg*gtt*$(v^\theta)^2$] kinetic energy (no coupling)

eke kinetic energy

emenc = r*denseq*[sqg*grr*$(B^\rho)^2$ + sqg*gtt*$(B^\theta)^2$] magnetic energy (no coupling)

eme magnetic energy

gampsi = $\Delta emenc_+ / (dt\ \Delta emenc_-)$ emenc growth rate

gamchi = $\Delta eme_+ / (dt\ \Delta eme_-)$ eme growth rate

gamlamb = $\Delta ekenc_+ / (dt\ \Delta ekenc_-)$ ekenc growth rate

gamalpha = $\Delta eke_+ / (dt\ \Delta eke_-)$ eke growth rate

gamphi = $\Delta eprnc_+ / (dt\ \Delta eprnc_-)$ eprnc growth rate

gampr = $\Delta epr_+ / (dt\ \Delta epr_-)$ epr growth rate

rint,xint,bb,cc,dd,xinte,bbe,cce,dde dummy vectors

lincheck & linstart

ivar variable index

nvar total number of variables

epsq = eps*eps

oneos = 1.0_IDP/s

betfc = bet0/(2.*epsq)

betfc_f = LcA2*bet0_f/(2.*epsq)

omcyd = omcy cyclotron frequency

grwth_avg averaged growth rate

omega_r_avg averaged real frequency

sum_iter iteration index

betfc_alp = LcA2alp*bet0_alp/(2.*epsq)

omcydalp = omcyalp cyclotron frequency second EP population

beteom = dpres*bet0/(2*epsq*omcyd)

betiom = (1-dpres)*bet0/(2*epsq*omcyd)

coef dummy real

deviation_freq frequency deviation regarding averaged frequency

deviation_growth growth rate deviation regarding averaged value

slhs = ytrhs(l,j)*yt(l,j) eq right hand side * left hand side

srhs = yt(l,j)**2 eq left hand side * left hand side

grwth growth rate

omega_r real frequency

ytrhs eq right hand side

vmec

j,l,l1,lq,lb0,m,m2,mm1,mm2,mm12,mm22,mm2p1,mm2p12,mp2,mp22,idummy,mband,n,nnn,mmm,lbm2,ir,jp,lp,nbst dummy integer

mjeqp number of flux surfaces (equilibrium input file)

mjeqm1 = mjeq-1

isg sign equilibrium poloidal mode

bigrn R coordinate (equilibrium input file)

pror pressure at the magnetic axis (equilibrium input file)

dummy dummy real

bet0in beta at the magnetic axis

twopi = 2*pi

twophip = 2.0*phip(mjeq)

one unit

gc = sqgeq

iota = 1/q

iotap iota at the magnetic axis

qmin minimum iota profile

qmax maximum iota profile

xl = r - rsb

llc,lls mode list

lbst beta star

rbinv inverse of the radial coordinate (equilibrium input file)

pfar pressure (equilibrium input file)

phip electrostatic potential (equilibrium input file)

curfar toroidal current (equilibrium input file)

ffar poloidal current (equilibrium input file)

sfar1,sfar2,sfar3,sfar4 dummy vector

rmnb radial coordinate (equilibrium input file)

sqgib,sqgb,grrb,grtb,gttb,bmodb,grrojb,grtojb,gttojb,jbgrrb,jbgrtb,jbgttb,sqgieq,sqgeq,grreq,grteq,gtteq,grzeq,gtzeq,gzzeq,jbgrreq,jbgrteq,jbgtteq,jbgrzeq,jbgtzeq,grrojeq,grtojeq,gttojeq,grzojeq,gtzojeq,gzzojeq,grrupeq,grtupeq,grzupeq,gttupeq,gtzupeq,gzzupeq,dbsjtojeq,dbsjzojeq,dbsjtbjeq,dgttreq,dgrrteq,dgrtteq,dgttteq,dgrrzeq,dgrtzeq,dgttzeq,dgrtpeq,dgttpeq,jsqeq,bsgrteqbsgtteq,bsqeq,bsqgtteq,sqgdrojeq,sqgdthojeq,sqgdztojeq,sqgdthojbsteq,sqgdztojbsteq,sqgibmoditheq,sqgibmodizteq, djrojeq,djtojeq,djzojeq, testeq,testreq,testteq,testrreq,testtteq,testrteq metric components and variables linked to metric component (equilibrium version)

bmodeq magnetic field (equilibrium version)

bsteq beta star (equilibrium version)

omdreq,omdteq,omdzeq Omega d operator components (equilibrium version)

omdrprpeq,omdtprpeq,omdzprpeq Omega d operator components for trapped EP (equilibrium version)

sb1,sb2,sb3,sb4,sb5,sb6,sb7 dummy vectors

lplrreq,lplrteq,lplrzeq,lpltteq,lpltzeq,lplzzeq,lplreq,lplteq,lplzeq perpendicular gradient operator terms (equilibrium version)

eildreq,eildteq,eildzeq,eildrreq,eildrteq,eildrzeq,eildtteq,eildtzeq,eildzzeq electron-ion Landau damping terms

bmodieq inverse magnetic field (equilibrium version)

bdceq = bmodeq(:,l)/(feq-qqinv*cureq)

brtdceq = bmodeq(:,l)*r*qqinv/(feq-qqinv*cureq)

bbbdceq = bmodeq*bmodeq*bmodeq /((feq-qqinv*cureq)*(feq-qqinv*cureq))

bbdceq = bmodeq*bmodeq/((feq-qqinv*cureq)*(feq-qqinv*cureq))

zetai,zetae,zetai2,zetai3,zetai4,zi,y0i,y1i,y2i,ddii,zetae2,zetae3,zetae4,ze,y0e,y1e,y2e,ddee,rei,sei,stfe,stfi,reii,sei1,sei2,sei3,zetaiinv,zetaeinv,cmplx1,xnuion0, xkprl, abkprl, sgkprl, xsii, xsie, xsi2, xsi3, xsi4,ztr, zti, zir, zii, zer, zei variables Landau damping elements

vther = vthe*sqrt(te)

vthir = vthi*sqrt(ti)

tauie = tinn/tenn

xnuelc = xnuelc0*dni/(te*sqrt(te))

xnuion = xnuion0*dni/(ti*sqrt(ti))

X. References

[1] Garcia, L., Proceedings of the 25th EPS International Conference, Prague, 22A, Part II, 1757, (1998)

[2] Charlton, L. A. et al, Journal of Comp. Physics, 63, 107, (1986).

[3] Charlton, L. A. et al, Journal of Comp. Physics, 86, 270, (1990).

[4] Spong, D. A. et al, Phys. Fluids B, 4, 3316, (1992).

[5] Hedrick, C. L. et al, Phys. Fluids B, 4, 3869, (1992).

[6] Spong, D. A. et al, Nucl. Fusion, 53, 053008, (2013).

[7] Hirshman, S. P. et al, Phys. Fluids, 26, 3553, (1983).

[8] Hirshman, S. P. et al, Phys. of Plasmas, 18, 062504, (2011).

[9] Lao L.L., et al, Nucl. Fusion, 25, 1611, (1985).

[10] Hammett, G. W. et al, Phys. Rev. Lett, 64, 3019, (1990).

[11] Boozer, A.H., Phys. Fluids, 25, 520, (1982).

[12] Varela, J. et al, Nucl. Fusion, 57, 046018, (2017).

[13] Varela, J. et al, Phys. Plasma, 19, 082501, (2012).

[14] Varela, J. et al, Phys. Plasma, 19, 082512, (2012).

[15] Varela, J. et al, Phys. Plasma, 21, 032501, (2014).

[16] Varela, J. et al, Phys. Plasma, 21, 092505, (2014).

[17] Varela, J. et al, Nucl. Fusion, 57, 126019, (2017).

[18] D. C. Pace, et al, Phys. Plasmas 25, 056109, 2018.

[19] J. Varela, et al, Nucl. Fusion, 58, 076017, 2018.

[20] J. Varela, *et al*, Nucl. Fusion, 59, 046008 (2019).

[21] J. Varela, et al, Nucl. Fusion, 59, 046017 (2019).

[22] S. Taimourzadeh et al, Nucl. Fusion, 59, 066006 (2019).

[23] J. Varela, et al, Phys. Plasmas, 26, 062502 (2019).

[24] J. Varela, et al, Nucl. Fusion, 59, 076036 (2019).

[25] J. Varela, et al, Nucl. Fusion, 60, 026016 (2020).

[26] J. Varela, *et al*", Nucl. Fusion, 60, 046013 (2020).

[27] S. Yamamoto, et al, Nucl. Fusion, 60, 066018 (2020).

[28] J. Varela, *et al*, Nucl. Fusion, 60, 096009 (2020).

[29] J. Varela, et al, Nucl. Fusion, 60, 112015 (2020).